Documentation

Major Functions and how they work:
DTSC-3601-Final-Project/INITALDEMOFILESHERE/.ipynb_checkpoints

CountsExtraction_Demo-checkpoint.ipynb

fetch_annual_traffic_counts():
Fetches annual traffic counts from the NCDOT API for specified location IDs only. Sends a GET request and checks for errors. Turns the JSON response into a DataFrame. Changes feature names to not include “attribute”.

Args: None, the IDs are pre-specified in the function.

Returns: ncdot_df: A DataFrame containing traffic data (NCDOT average annual traffic counts) for the specified locations.

save_traffic_counts_to_s3(ncdot_df):
Saves the traffic counts DataFrame to an S3 bucket in CSV format with the current timestamp. Confirms if data is sent to the S3 bucket and outputs where the file can be found (file key).
Args: ncdot_df: A DataFrame containing traffic data (NCDOT average annual traffic counts).
Returns: None, but prints the saved file with the associated timestamp

SpeedsExtraction_Demo-checkpoint.ipynb

fetch_tomtom_traffic_data():
Fetches traffic data from the TomTom API for specified latitude and longitude coordinates and returns a DataFrame containing the relevant information. Extracts the current speed, free-flow speed, current delay, and confidence score from each successful response. Adds successful pulls to “data_list” and prints an error if unsuccessful.

Args: None; the latitude and longitude coordinates are hardcoded in the function.

Returns: tomtom_df (DataFrame): A DataFrame containing the traffic data for the specified coordinates, including timestamp, latitude, longitude, current speed, free flow speed, current delay, and confidence

edit_time_features(tomtom_df):

Adds time features to the TomTom traffic DataFrame; including hour, day of the week, month, and speed ratio as properties of existing features (timestamp and current_speed vs. free_flow_speed). Monday is 0 and Sunday is 6.

Args: tomtom_df (DataFrame): A DataFrame containing the traffic data.
Returns: tomtom_df (DataFrame): The DataFrame with the added features. speed_ratio is calculated as current_speed divided by free_flow_speed. The time features are extracted from the timestamp and include hour, day_of_week, and month

add_holiday_info(tomtom_df):
Adds holiday information to the TomTom traffic DataFrame by mapping timestamps to US federal holidays. Labeled dates that do not match federal holidays are listed as “Not a federal holiday”

Args: tomtom_df (DataFrame): A DataFrame containing the traffic data.

Returns: tomtom_df (DataFrame): The DataFrame with an added 'holiday' column indicating whether each timestamp falls on a US federal holiday or not

save_tomtom_data_to_s3(tomtom_df):
Saves the traffic information DataFrame to an S3 bucket in CSV format with the current timestamp.

Args: tomtom_df: A DataFrame containing traffic data (NCDOT average annual traffic counts).

Returns: None

Inital_Demo-checkpoint.ipynb

extract_tomtom_traffic_data():
Extracts traffic data from the specified S3 bucket and prefix, decompresses the files if necessary, and combines them into a single DataFrame. Initializes an empty list to store the DataFrames created from each file. File contents are read as raw bytes.
Args: None

Returns: tomtom_df: pd.DataFrame: A DataFrame containing the combined traffic data from all relevant files in the S3 bucket.

school_holiday_check(traffic_df):
Checks if a given date falls within the defined school holiday periods, including the start and end dates. Dates were manually put in to match the UNC Charlotte school calendar.

Args: traffic_df (pd.DataFrame): A DataFrame containing traffic data with a 'timestamp' column to be compared to school holiday periods.

Returns: traffic_df (pd.DataFrame): The input DataFrame with an additional 'school_holiday' column indicating whether each date falls within a school holiday period. 1 for holiday, 0 for non-holiday. For days of the week, Monday = 0, Sunday = 6.

extract_ncdot_traffic_data_s3():
Extracts traffic data from the specified S3 bucket and key for NCDOT traffic data, decompresses the file if necessary, and returns it as a DataFrame with coordinates edited to match the other data (account for slight decimal differences). Creates matched_latitude and matched_longitude columns using the predefined latitude and longitude values. Matches the coordinates to each row by position, assuming the NCDOT records remain in the expected order.

Args: None

Returns: ncdot_df: pd.DataFrame: A DataFrame containing the NCDOT traffic data.

extract_ncdot_traffic_data(ncdot_df):
For a provided DataFrame of NCDOT traffic data, edits the coordinates to match the TomTom data perfectly (account for slight decimal differences). Pared down to only the coordinate matching part of the S3 extraction function, so it can be used on any DataFrame of NCDOT traffic data.

Args: ncdot_df (pd.DataFrame): A DataFrame containing the NCDOT traffic data.

Returns: ncdot_df (pd.DataFrame): The input DataFrame with updated coordinates to match the TomTom data.

combine_dataframes(tomtom_df, ncdot_df): Combines two DataFrames (tomtom_df and ncdot_df) by matching latitude and longitude values, and adds the AADT_2022 column from ncdot_df to tomtom_df. Creates a temporary round_key column in the NCDOT DataFrame by combining the rounded latitude and longitude values into a single lookup key. Creates a lookup table that maps each coordinate key to its

corresponding AADT_2022 value. Assigns the matched AADT values to a new AADT_2022 column in the TomTom DataFrame. Unmatched coordinates receive missing values (NaN). Then removes the temporary round key columns from both DataFrames.

Args:
tomtom_df: The first DataFrame containing traffic data with latitude and longitude columns.
ncdot_df: The second DataFrame containing traffic data with matched_latitude, matched_longitude, and AADT_2022 columns.

Returns: traffic_df (pd.DataFrame): A combined DataFrame with the AADT_2022 column added from ncdot_df to tomtom_df.

preprocess_train_data(combined_df):
Preprocesses the combined DataFrame by cleaning the data, creating new features, and preparing it for machine learning. Trains an XGBoost regression model on the preprocessed data. Sorts data chronologically (oldest to newest). 1 = US federal holiday, 0 = Not a US federal holiday. Defines X and y, using current_speed as the value to predict. Splits the data into an 80% training, 20% testing split. Trains an XGBoost regression model with 100 trees, a maximum depth of 5, and a learning rate of 0.1.

Args: combined_df (pd.DataFrame): The combined DataFrame containing all the data.

Returns: model (xgb.XGBRegressor): The trained XGBoost regression model.

evaluate_model(model, X_test, y_test):
Evaluates the performance of the trained model on the test set and prints various performance metrics. Calculates MAE, RMSE, MAPE, and R^2. Replaces actual speeds of zero with a small value when calculating MAPE to avoid division by zero.

Args:
- model: xgb.XGBRegressor: The trained XGBoost regression model.
- X_test (pd.DataFrame): The test set features.
- y_test (pd.Series): The actual target values for the test set.

Returns: None; it displays the evaluation metrics and predicted outputs for some inputs in the test set.

predict_custom_traffic(model, location_id, timestamp_str):

Takes a single location_id and timestamp string, automatically assigns the hardcoded free_flow_speed and AADT_2022, runs feature engineering, and predicts traffic speed and congestion level. Calls school_holiday_check() to determine whether the timestamp falls during a designated school holiday. Creates the is_holiday feature using the school holiday indicator. Selects the required features in the same order used to train the model. Uses the trained XGBoost model to predict the traffic speed. Calculates the speed index by dividing the predicted speed by the location's free-flow speed. Classifies traffic conditions based on the speed index:
- Low Traffic: Speed index ≥ 0.8
- Medium Traffic: Speed index ≥ 0.5 but < 0.8
- High Traffic: Speed index < 0.5

Args:
model: xgb.XGBRegressor: The trained XGBoost regression model.
location_id (int): The location ID for which to predict traffic speed.
timestamp_str (str): The timestamp string in the format 'YYYY-MM-DD HH:MM:SS' for which to predict traffic speed.
Returns: tuple: A tuple containing the predicted traffic speed (float) and congestion level (str).